
Trabajo Grupal - Primer Parcial

Física Computacional

Nombres de los Integrantes:

Mamani Huarsaya, Jorge Luis

Aragón Carpio Fredy José

Estudiante 3

Estudiante 4

Estudiante 5

13 de mayo de 2026

Índice

1. Movimiento parabólico	3
1.1. Enunciado	3
1.2. Aplicación de la ecuación	3
1.3. Aplicación del método de Heun	5
1.4. Comparación de resultados	7
2. Simulación de 5 cuerpos gravitacionales	8
2.1. Modelo y aceleraciones	8
2.2. Implementación	8

1. Movimiento parabólico

1.1. Enunciado

La trayectoria de una partícula es (extraído del libro de Serway o Sears):

$$y = x \tan \alpha_0 - \frac{g}{2v_0^2 \cos^2 \alpha_0} x^2 \quad (1)$$

donde v_0 y α_0 son las condiciones iniciales.

1. Mediante una gráfica determine el alcance mediante la ecuación de arriba. Las condiciones iniciales son $v_0 = 5 \text{ m/s}$ con $\alpha_0 = 60^\circ$.
2. Aplicar el método de Heun y encuentre el alcance.
3. Haga la comparación de los resultados encontrados. Para ello varíe el tamaño de paso h para que el resultado sea confiable.

1.2. Aplicación de la ecuación

Para resolver la primera parte del problema, se utilizó directamente la ecuación analítica del movimiento parabólico mostrada en la Ecuación 1. Se implementó un programa en Python que genera múltiples valores de la posición horizontal x y evalúa la altura correspondiente $y(x)$ para cada punto.

Las condiciones iniciales consideradas fueron $v_0 = 5 \text{ m/s}$, $\alpha_0 = 60^\circ$ y $g = 10 \text{ m/s}^2$. Luego, se graficó la trayectoria de la partícula y se observó la forma parabólica característica del movimiento y determinar visualmente el alcance horizontal.

La implementación utilizada se muestra en el Código 1.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Parámetros
5 g = 10
6 v0 = 5
7 alpha = np.radians(60)
8
9 # Ecuación
10 x = np.linspace(0, 2.3, 500)
11 y = x * np.tan(alpha) - (g / (2 * v0**2 * np.cos(alpha)**2)) * x**2
12
13 # Encontrar el alcance
14 indice = np.where(y >= 0)[0][-1]
15 alcance = x[indice]
16
17 # Gráfica
18 plt.plot(x, y)
19 plt.scatter(alcance, 0)
20 plt.text(alcance, 0, f'{alcance:.4f}')
21 plt.grid(True)
22 plt.xlabel('x (m)')
23 plt.ylabel('y (m)')
24 plt.title('Movimiento parabólico - Analítico')
25
26 plt.savefig('../images/problem01_1.png', dpi=300, bbox_inches='tight')
27 plt.show()
```

Código 1: Implementación de la gráfica de la ecuación 1.

La Figura 1 muestra la trayectoria obtenida mediante la ecuación analítica.

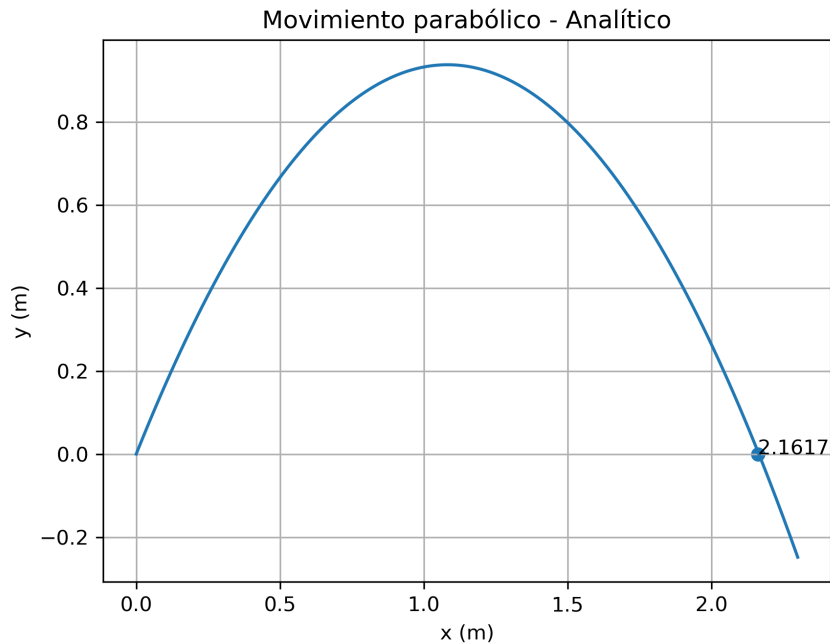


Figura 1: Solución analítica.

A partir de la gráfica obtenida, se observa que el proyectil alcanza aproximadamente una dis-

tancia horizontal de 2.16 m antes de regresar al suelo.

1.3. Aplicación del método de Heun

Para resolver numéricamente el movimiento parabólico, se empleó el método de Heun, el cual corresponde a un método predictor-corrector de segundo orden utilizado para aproximar soluciones de ecuaciones diferenciales ordinarias.

El movimiento de la partícula puede describirse mediante el siguiente sistema:

$$\frac{dx}{dt} = v_x \quad (2)$$

$$\frac{dy}{dt} = v_y \quad (3)$$

$$\frac{dv_x}{dt} = 0 \quad (4)$$

$$\frac{dv_y}{dt} = a \quad (5)$$

donde la aceleración gravitacional se considera constante:

$$a = -10 \text{ m/s}^2 \quad (6)$$

Las componentes iniciales de la velocidad se calcularon a partir de la velocidad inicial y el ángulo de lanzamiento:

$$v_{x0} = v_0 \cos \alpha_0 \quad (7)$$

$$v_{y0} = v_0 \sin \alpha_0 \quad (8)$$

El método de Heun se aplicó utilizando un esquema predictor-corrector. En cada iteración, primero se estimó una velocidad vertical predictora mediante el método de Euler y posteriormente se corrigió la posición vertical utilizando el promedio entre la pendiente inicial y la pendiente predicha.

La integración numérica se realizó para intervalos de tiempo separados por un tamaño de paso $h = 0.01$. La simulación continuó hasta que la partícula regresó al suelo, es decir, cuando la coordenada vertical tomó valores menores que cero.

La implementación desarrollada se muestra en el Código 2.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Parámetros
5 a = -10
6 v0 = 5
7 alpha = np.radians(60)
8
9 vx = v0 * np.cos(alpha)
10 vy = v0 * np.sin(alpha)
11
12 x = 0
13 y = 0
14
15 h = 0.01
16 tfin = 50
17
18 px = [x]
19 py = [y]
20
21 for t in np.arange(0, tfin, h):
22
23     vy_pred = vy + h * a
24
25     y = y + h * (vy + vy_pred) / 2
26     x = x + h * vx
27
28     vy = vy_pred
29
30     px.append(x)
31     py.append(y)
32
33     if y < 0:
34         break
35
36 # Último punto físico válido
37 alcance = px[-2]
38
39 # Gráfica
40 plt.plot(px, py)
41 plt.scatter(alcance, 0)
42 plt.text(alcance, 0, f'{alcance:.4f}')
43 plt.grid(True)
44 plt.xlabel('x (m)')
45 plt.ylabel('y (m)')
46 plt.title('Movimiento parabólico - Método de Heun')
47
48 plt.savefig('../images/problem01_2.png', dpi=300, bbox_inches='tight')
49 plt.show()
```

Código 2: Implementación del método de Heun para el movimiento parabólico.

La Figura 2 muestra la trayectoria obtenida mediante el método numérico.

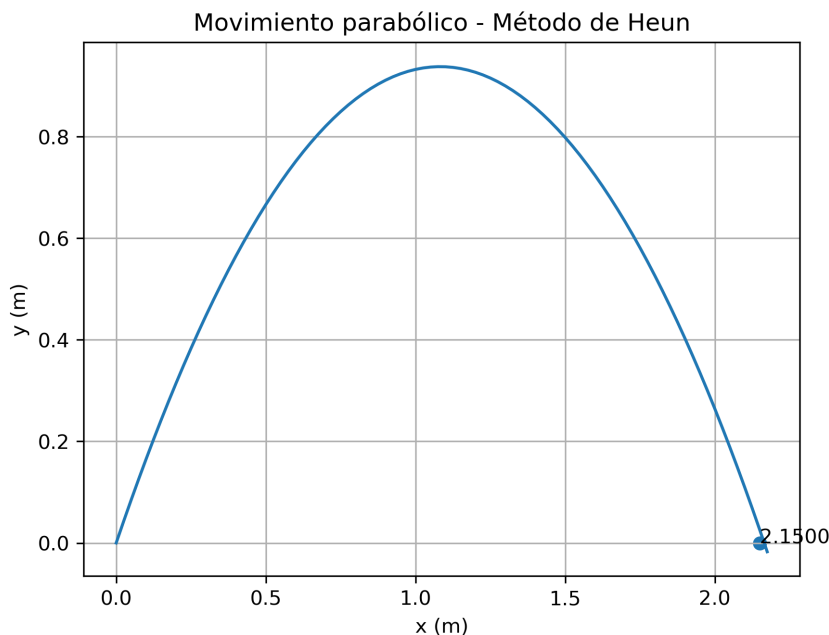


Figura 2: Trayectoria obtenida mediante el método de Heun.

El alcance horizontal obtenido mediante el método de Heun fue aproximadamente 2.16 m, mostrando una buena concordancia con el resultado analítico obtenido previamente.

1.4. Comparación de resultados

Finalmente, se realizaron simulaciones utilizando distintos tamaños de paso h con el objetivo de analizar la convergencia del método de Heun. Para cada simulación, el alcance horizontal se determinó tomando el último punto válido antes de que la trayectoria atravesase el suelo, es decir, antes de que la coordenada vertical tome valores negativos.

Posteriormente, los resultados numéricos fueron comparados con el alcance obtenido mediante la solución analítica desarrollada en la primera parte del problema.

Los resultados obtenidos se muestran en la Tabla 1.

Paso h	Alcance obtenido (m)	Error relativo (%)
0.1	2.0000	7.4812
0.01	2.1500	0.5423
0.001	2.1650	0.1516

Tabla 1: Comparación del alcance para diferentes tamaños de paso.

Se observa que el alcance calculado mediante el método numérico depende directamente del tamaño de paso utilizado durante la integración temporal. Para valores grandes de h , la simulación produce una aproximación menos precisa debido a que el método avanza mediante intervalos discretos relativamente amplios.

Al disminuir el tamaño de paso, los puntos calculados se aproximan progresivamente a la trayectoria analítica y el error relativo disminuye considerablemente. Esto evidencia la convergencia del método de Heun hacia la solución exacta del problema.

En conclusión, el método de Heun proporciona resultados satisfactorios para el movimiento pa-

rábólico y mejora su precisión conforme se reduce el tamaño de paso empleado en la simulación.

2. Simulación de 5 cuerpos gravitacionales

Simular el movimiento de 5 cuerpos gravitacionales. Los primeros 4 se ubican en los vértices de un cuadrado de lado a . Encontrar las aceleraciones, modificar las funciones de dichas aceleraciones y graficar las trayectorias resultantes.

2.1. Modelo y aceleraciones

El sistema consiste en cinco cuerpos que interactúan entre sí mediante la ley de gravitación. A diferencia de un sistema con un único centro de fuerza, cada cuerpo experimenta la atracción simultánea de los otros cuatro, lo que genera trayectorias altamente no lineales e impredecibles a simple vista.

La aceleración del cuerpo i producida por la influencia gravitacional de los demás cuerpos se expresa como:

$$\vec{a}_i = \sum_{j \neq i} \frac{Gm_j(\vec{r}_j - \vec{r}_i)}{(|\vec{r}_{ij}|^2 + \varepsilon^2)^{3/2}} \quad (9)$$

donde $\varepsilon = 0.15$ es un parámetro de suavizado que evita la divergencia numérica cuando dos cuerpos se aproximan excesivamente. Sin este término, la magnitud de la aceleración tendería a infinito al reducirse la distancia entre cuerpos, desestabilizando completamente la integración numérica.

Los cuatro cuerpos exteriores de masa $m = 1$ se ubican en los vértices de un cuadrado de lado $a = 2$, El cuerpo central, con masa $m_5 = 5$, ejerce una atracción cinco veces mayor que cada uno de los cuerpos exteriores. Esto implica que los cuatro cuerpos en las esquinas sienten una fuerza dominante hacia el centro, mientras que las interacciones entre ellos generan perturbaciones laterales que desvían sus trayectorias del movimiento circular ideal. El resultado es un sistema caótico donde pequeñas variaciones en las condiciones iniciales o en el tamaño de paso producen trayectorias cualitativamente distintas.

2.2. Implementación

La implementación se organiza en tres bloques. El primero define los parámetros físicos del sistema: la constante gravitacional $G = 1$, las masas, el lado del cuadrado $a = 2$, el parámetro de suavizado ε y el paso de integración $h = 0.002$ con tiempo final $t_{\text{fin}} = 15$. Luego se establecen las posiciones iniciales de los cinco cuerpos y sus velocidades tangenciales, calculadas a partir de la velocidad orbital $v_{\text{orb}} = 0.80\sqrt{GM_{\text{total}}/R}$ con $R = a/\sqrt{2}$, de modo que el sistema parta con cierta inercia rotacional. Finalmente se inicializan los arreglos que almacenarán las trayectorias a lo largo de la simulación.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 # Parámetros del sistema
4 G = 1.0
5 m = 1.0
6 m5 = 5.0
7 a = 2.0
```

```

8  eps  = 0.15
9  h    = 0.002
10 tfin = 15

```

```

27 # Condiciones iniciales
28 # C1:(+a/2,+a/2) C2:(-a/2,+a/2)
29 # C3:(-a/2,-a/2) C4:(+a/2,-a/2) C5:(0,0)
30 masas = np.array([m, m, m, m, m5])
31 N      = 5
32
33 rx = np.array([ a/2, -a/2, -a/2,  a/2,  0.0])
34 ry = np.array([ a/2,  a/2, -a/2, -a/2,  0.0])
35
36 R    = a / np.sqrt(2)
37 v_orb = 0.80 * np.sqrt(G * np.sum(masas) / R)
38
39 vx = np.array([-v_orb,  v_orb,  v_orb, -v_orb,  0.0])
40 vy = np.array([ v_orb,  v_orb, -v_orb, -v_orb,  0.0])
41
42 # Inicializar arreglos
43 nmax = int(round(tfin / h)) + 2
44 prx  = np.zeros((N, nmax))
45 pry  = np.zeros((N, nmax))
46
47 n = 0
48 prx[:, 0] = rx
49 pry[:, 0] = ry

```

Código 3: Parámetros del sistema, condiciones iniciales e inicialización de arreglos.

El segundo bloque contiene la función `aceleraciones`, que recibe las posiciones actuales de todos los cuerpos y calcula la aceleración neta sobre cada uno aplicando la ecuación gravitacional con suavizado. Seguido del bucle de integración mediante el método de Euler, donde en cada paso se evalúan las aceleraciones, se actualizan las velocidades y posiciones, y se registran las coordenadas en los arreglos de trayectoria.

```

12 # Función de aceleraciones
13 def aceleraciones(rx, ry, masas, G, eps):
14     N = len(masas)
15     ax = np.zeros(N)
16     ay = np.zeros(N)
17     for i in range(N):
18         for j in range(N):
19             if i != j:
20                 dx = rx[j] - rx[i]
21                 dy = ry[j] - ry[i]
22                 r3 = (dx**2 + dy**2 + eps**2)**1.5
23                 ax[i] = ax[i] + G * masas[j] * dx / r3
24                 ay[i] = ay[i] + G * masas[j] * dy / r3
25     return ax, ay

```

```

51 # Método de Euler
52 for t in np.arange(0, tfin - h, h):
53     n += 1
54     ax, ay = aceleraciones(rx, ry, masas, G, eps)
55     vx = vx + ax * h
56     vy = vy + ay * h
57     rx = rx + vx * h

```

```

58     ry = ry + vy * h
59     prx[:, n] = rx
60     pry[:, n] = ry
61
62 prx = prx[:, :n+1]
63 pry = pry[:, :n+1]

```

Código 4: Función de aceleraciones gravitacionales y método de Euler.

El tercer bloque genera la figura de trayectorias con fondo oscuro. Cada cuerpo se representa con un color distinto: la trayectoria completa se traza con baja opacidad como estela, y el último 25 % del recorrido se resalta con mayor brillo para mostrar la dirección de movimiento reciente. Los marcadores circulares indican la posición inicial y las estrellas la posición final de cada cuerpo.

```

65 # FIGURA
66 colores = [
67     [0.2, 0.5, 1.0],
68     [1.0, 0.3, 0.3],
69     [0.2, 0.9, 0.4],
70     [1.0, 0.7, 0.1],
71     [0.8, 0.3, 1.0]
72 ]
73 nombres = ['C1 (+,+)', 'C2 (-,+)', 'C3 (-,-)', 'C4 (+,-)', 'C5 centro']
74
75 fig, ax1 = plt.subplots(figsize=(12, 10))
76 fig.patch.set_facecolor([0.05, 0.05, 0.08])
77 ax1.set_facecolor([0.05, 0.05, 0.08])
78 ax1.tick_params(colors=[0.5, 0.5, 0.5])
79 ax1.xaxis.label.set_color([0.75, 0.75, 0.75])
80 ax1.yaxis.label.set_color([0.75, 0.75, 0.75])
81 for spine in ax1.spines.values():
82     spine.set_edgecolor([0.25, 0.25, 0.25])
83 ax1.grid(True, color=[0.25, 0.25, 0.25], alpha=0.5)
84 ax1.set_aspect('equal')
85
86 for i in range(N):
87     c = colores[i]
88     # Estela tenue
89     ax1.plot(prx[i, :], pry[i, :], '-',
90             color=c + [0.4], linewidth=1.0)
91     # Cola brillante: último 25%
92     k0 = int(round(0.75 * n))
93     ax1.plot(prx[i, k0:], pry[i, k0:], '-',
94             color=c + [1.0], linewidth=2.0, label=nombres[i])
95
96 for i in range(N):
97     ax1.plot(prx[i, 0], pry[i, 0], 'o',
98             markersize=9, markerfacecolor=colores[i],
99             markeredgecolor='k', markeredgewidth=1.2)
100
101 for i in range(N):
102     ax1.plot(prx[i, -1], pry[i, -1], 'p',
103             markersize=12, markerfacecolor=colores[i],
104             markeredgecolor='k', markeredgewidth=1.0)
105
106 ax1.set_xlabel('x')
107 ax1.set_ylabel('y')
108 ax1.set_title('Trayectorias - 5 Cuerpos Gravitacionales',
109             color='w', fontsize=14)
110 ax1.legend(loc='upper right')
111

```

```

112 plt.tight_layout()
113 plt.savefig('images/problem03.png', dpi=150,
114             facecolor=fig.get_facecolor(), bbox_inches='tight')
115 plt.show()

```

Código 5: Configuración visual y generación de la figura de trayectorias.

La Figura 3 muestra las trayectorias obtenidas para los cinco cuerpos durante el tiempo de simulación $t_{\text{fin}} = 15$.

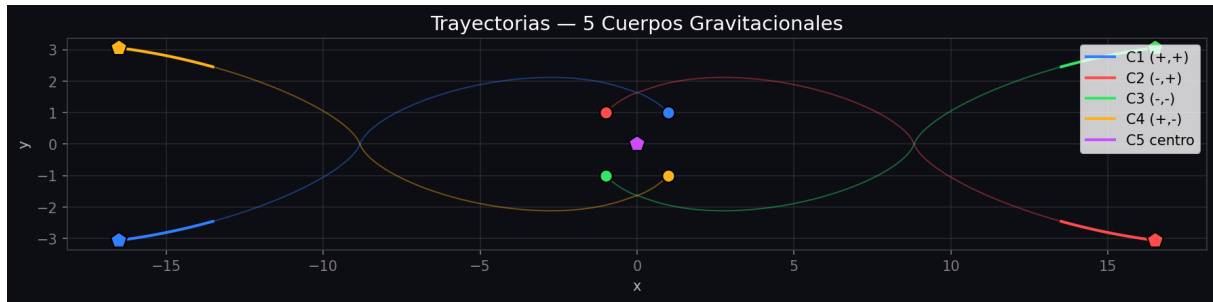


Figura 3: Trayectorias de los 5 cuerpos gravitacionales obtenidas mediante el método de Euler. Los círculos indican posiciones iniciales y las estrellas las posiciones finales.

Se observa que los cuatro cuerpos exteriores no describen órbitas cerradas sino trayectorias abiertas y asimétricas. La atracción del cuerpo central domina el movimiento global, actuando como un núcleo gravitacional que curva continuamente las trayectorias de los cuerpos periféricos. Sin embargo, las interacciones mutuas entre estos últimos generan aceleraciones transversales que rompen la simetría inicial del sistema, produciendo el comportamiento caótico visible en la figura. Los cuerpos que parten de posiciones diagonalmente opuestas tienden a alejarse en direcciones contrarias, evidenciando que el sistema no conserva la simetría rotacional más allá de los primeros instantes de la simulación.